

A-Life Pac-Man

Ecosystem Architecture, Evolutionary Neural Networks, and Decoupled Agency

Prepared For: Advanced Agentic Coding Research Review

Author: Antigravity AI Engineering Division

Date: May 2026

Status: Published Technical Guide

Document Version: 1.4.0

Table of Contents

Chapter 1: Executive Overview & Simulation Core	Page 3
Chapter 2: PyTorch Neural Network Brain (brain.py)	Page 4
Chapter 3: Cognitive Control & Instinct Drives (soul.py)	Page 5
Chapter 4: Physical Bodies, Development & Locomotion (entity.py)	Page 6
Chapter 5: World Simulation Grid, Navigation & Social Systems (world.py)	Page 7
Chapter 6: Dynamic Vector Rendering (ui.py)	Page 8

Chapter 1: Executive Overview & Simulation Core

The A-Life Pac-Man project is a bio-inspired evolutionary neural network simulation that merges visual game mechanics with complex cognitive agent behaviors. Unlike traditional Pac-Man where entities rely on deterministic state machines or static pathing trees, this system models two competing species ('spots' and 'stripes') whose behaviors are governed by independent feedforward neural networks. Over successive generations, these weights undergo selection pressure based on foraging efficiency, base defense, and mate attraction.

The primary design pillars of the simulation include:

- **Decoupled Architecture:** Physical simulation loops, visual render pipelines, and agent mind controllers ('souls') are strictly decoupled. This allows headless execution, independent testing, and modular extensions.
- **Genetic Evolution:** A double-pass crossover mechanism combined with gaussian mutation operates directly on neural parameter weights. Fitness points are dynamically awarded based on productive tasks (storing food, defending bases, reproducing).
- **Day/Night & Curse Cycle:** Environmental variations force temporal behaviors. Day cycles focus on productivity and foraging. Night cycles trigger zombie invasions, introducing survival pressures and curfew enforcement.
- **Hardware Acceleration:** Tensors representing neural connections are mapped directly onto accelerated system frameworks (such as Apple Silicon's MPS or NVIDIA CUDA) via PyTorch.

Chapter 2: PyTorch Neural Network Brain (brain.py)

The **Brain** class represents the cognitive hardware of an entity. It is structured as a two-layer fully connected feedforward neural network with 7 input nodes, 12 hidden nodes, and 6 output nodes. Weights are stored as PyTorch float32 tensors, enabling accelerated tensor multiplications.

Sensory Inputs (7 dimensions):

1. *Proximity to Food*: $1.0 / (1.0 + \text{distance to nearest mushroom or worm})$.
2. *Proximity to Castle*: $1.0 / (1.0 + \text{distance to species' home base})$.
3. *Proximity to Enemy*: Proximity value to closest adult of the opposite species.
4. *Proximity to Mate*: Proximity value to closest compatible mate with inactive mating cooldown.
5. *Energy Level*: Current physical energy scaled [0.0 - 1.0].
6. *Age Ratio*: Current lifespan relative to world life expectancy.
7. *Reserves Level*: Current food reserves stored in the home base scaled by maximum storage capacity.

Output Actions (6 dimensions):

The output vector uses a *tanh* activation function, representing drive strength [-1.0, 1.0] for the six actions: **Forage**, **Store**, **Attack**, **Mate**, **Cultivate (Plant)**, and **Defend (Guard)**.

The neural feedforward operation maps inputs directly onto device weights using PyTorch:

```
class Brain:
    def __init__(self, input_size=7, hidden_size=12, output_size=6, weights=None):
        self.input_size = input_size
        self.hidden_size = hidden_size
        self.output_size = output_size

        if weights is not None:
            self.w1 = weights['w1'].clone().to(DEVICE)
            self.w2 = weights['w2'].clone().to(DEVICE)
        else:
            self.w1 = (torch.rand((input_size, hidden_size), dtype=torch.float32, device=DEVICE) * 2.0 - 1.0)
            self.w2 = (torch.rand((hidden_size, output_size), dtype=torch.float32, device=DEVICE) * 2.0 - 1.0)

    def feed_forward(self, inputs_list):
        inputs_tensor = torch.tensor(inputs_list, dtype=torch.float32, device=DEVICE)
        hidden = torch.tanh(torch.matmul(inputs_tensor, self.w1))
        outputs = torch.tanh(torch.matmul(hidden, self.w2))
        return {
            'hidden': hidden.cpu().numpy(),
            'outputs': outputs.cpu().numpy()
        }
```

Chapter 3: Cognitive Control & Instinct Drives (soul.py)

The **Soul** acts as the decoupled controller of the physical Pac-Man body. It polls the physical coordinates and the world environment, compiles the 7 normalized inputs, queries the `Brain` feedforward network, and resolves drives. It also applies crucial biological and environmental rules that override neural defaults during critical states.

Heuristic Instinct Overrides:

- **Virgin Mating Drive:** Young adults who have not yet passed on their genetics receive a temporary mating drive boost (+0.85) to encourage genetic continuation.
- **Female Protection:** Males near friendly females who are threatened by rival males have their Attack output boosted (+1.5) to engage in defense.
- **Extinction Avoidance:** If a species' population falls below 3, mating drives are heavily boosted (+1.5) to prevent total collapse.
- **Resource Delivery Lock:** If carrying food, carrying a baby, or carrying a worm, the active drive is locked to 'Store' to ensure base safety and development.

The drive decision logic from `soul.py` operates as follows:

```
# Choose highest output drive
drives = ["Forage", "Store", "Attack", "Mate", "Cultivate", "Defend"]
max_val = -float('inf')
max_drive_index = 0

for i, drive in enumerate(drives):
    val = outputs[i]
    # Restriction on infants and old age: no attack (2), mate (3), or cultivate (4)
    if body.is_infant() or body.is_old():
        if i in [2, 3, 4]:
            continue
    if val > max_val:
        max_val = val
        max_drive_index = i

body.current_drive = drives[max_drive_index]

# Force 'Store' drive under carrying rules
if body.food_carried > 0 or getattr(body, 'has_worm', False) or getattr(body, 'carrying_baby', None) is not None or body.is_infant():
    if body.current_drive not in ["Store", "Cultivate"]:
        body.current_drive = "Store"
```

Chapter 4: Physical Bodies, Development & Locomotion (entity.py)

Physical simulation metrics are maintained inside the `Pacman` class in `entity.py`. A Pacman possesses position coordinates, velocity glide factors, energy pools, and developmental age brackets. Unlike basic grid systems where entities instantly teleport between tiles, Pacman entities glide smoothly, calculating intermediate coordinate states per simulation tick.

Developmental Stages:

- **Infancy:** Less than 10% of total lifespan. Cannot mate, plant, or attack. Energy loss is reduced, and they automatically feed from home base reserves to grow.
- **Adulthood:** 10% to 90% of lifespan. Capable of all drives. Energy loss is standard.
- **Senescence (Old Age):** Greater than 90% of lifespan. Speed is reduced by 40%. Cannot mate or plant. Typically act as castle defenders, blocking gates against hostile thieves.

Locomotion speed modifiers apply based on current state (e.g. holding cargo slows movement by 15%):

```
def move(self):
    if self.carried_by is not None:
        self.x = self.carried_by.x
        self.y = self.carried_by.y
        return

    if not self.target_tile:
        return

    dx = self.target_tile['x'] - self.x
    dy = self.target_tile['y'] - self.y
    dist = math.sqrt(dx*dx + dy*dy)

    # Base speed & modifiers
    current_speed = self.speed
    if self.is_old():
        current_speed = self.speed * 0.6
    elif self.is_infant():
        current_speed = self.speed * 0.65

    if self.food_carried > 0 or self.has_worm:
        current_speed *= 0.85

    if dist <= current_speed:
        self.x = float(self.target_tile['x'])
        self.y = float(self.target_tile['y'])
        self.tile_x = self.target_tile['x']
        self.tile_y = self.target_tile['y']
        self.check_target_interaction()

        if len(self.current_path) > 0:
            self.target_tile = self.current_path.pop(0)
        else:
            self.target_tile = None
    else:
        self.x += (dx / dist) * current_speed
        self.y += (dy / dist) * current_speed
```

Chapter 5: World Grid, Navigation & Social Systems (world.py)

The `**SimulationWorld**` manages grid coordinates, resource spawning, time-of-day cycles, and physical interaction resolvers (combat, mating, mentoring). The grid maps a custom 47x21 layout containing a structured maze (columns 0-26), a partitioned division wall with gates (column 27), and a wild forest populated with trees and worms (columns 28-46).

Interaction Resolvers:

- **Social Mentoring:** When an adult and infant of the same species occupy adjacent cells, they initiate mentoring. The infant's brain tensors are updated on the PyTorch device via a linear blending interpolation with the adult's weights, accelerating learning.
- **Territorial Combat:** Adult males of opposing species that cross paths engage in combat. The winner's fitness is boosted, and the loser is removed. If a senior defender blocks a base gate, they stall hostile adults, draining the attacker's energy.
- **Agricultural Cultivation:** Once a base accumulates 20 mushrooms, 'Agricultural Cultivation' unlocks. Adults carrying food can seed nearby garden tiles to cultivate crops, reducing dependency on wild forage.

The weight blending mechanism for mentoring operates directly on GPU/MPS acceleration frameworks:

```
def mentor_infant(self, adult, infant):
    adult.mentor_cooldown = 15.0
    infant.mentor_cooldown = 15.0

    # Blend neural parameters directly on active PyTorch device (MPS/CPU)
    with torch.no_grad():
        infant.brain.w1 = infant.brain.w1 * 0.85 + adult.brain.w1 * 0.15
        infant.brain.w2 = infant.brain.w2 * 0.85 + adult.brain.w2 * 0.15

    adult.fitness += 1
    adult.thought = f"Teaching {infant.name} how to survive and forage."
    infant.thought = f"Mentored by {adult.name}! Feeling wiser."

    if infant.energy < 40 and adult.food_carried > 0:
        adult.food_carried -= 1
        infant.energy = min(100.0, infant.energy + 40.0)
```


Chapter 6: Dynamic Vector Rendering (ui.py)

The `**SimulationUI**` class handles real-time rendering. It draws the active game grid, base enclosures, and developmental indicators (carried mushrooms, babies, or forest worms). Beyond grid graphics, the UI features three diagnostic overlays designed for evolutionary monitoring:

1. Neural Brain Synapse Graph

A live visualization of the inspected Pac-Man's neural network. It maps the 7 inputs, 12 hidden nodes, and 6 outputs. Weight lines (synapses) are colored based on charge (orange for positive, cyan for negative) with line thickness proportional to connection strength.

2. Population History Plot

Plots real-time population sizes of both species over the last 30 intervals, helping engineers visualize population booms, competitive exclusions, and extinction events.

3. System Event Log Overlay

Displays a running terminal-style console of global events (births, deaths, theft, invasions, eras) at the bottom of the sidebar HUD for instant debugging.

Closing Notes

This architectural setup demonstrates how complex, organic-like behaviors can emerge from simple feedforward neural inputs when paired with spatial constraints and localized reward mechanics. The modularity of the decoupled classes ensures that further extensions (like multi-layer brains, deeper cognitive nodes, or varied map structures) can be integrated without breaking core physics.